Taller de Python
Construyendo tu Tamagotchi-Bot desde cero

IEEE RAS Student Branch Chapter

Universidad de Concepción

18 de Marzo, 2026

Temario del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

IEEE Robotics & Automation Society

Hoja de Ruta del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

IEEE Robotics & Automation Society

Entorno y Salida de datos (Output): `print()`

Usaremos Google Colab, que ejecuta nuestro código en la nube sin instalar nada. Para comunicarnos con el entorno, usamos `print()`.

```python
print("Iniciando sistemas del robot...")
print("Hello World!")
```

Entrada de datos (Input): `input()`

Para que el programa reciba comandos desde nuestro teclado, usamos `input()`. **El programa se pausará** hasta que escribamos algo y presionemos Enter.

```
input("Presiona ENTER para iniciar el motor principal:
    ")
```

Variables: La memoria del robot

Una variable es un espacio en la memoria **con un nombre** donde guardamos datos. Usamos el signo igual (=) para asignar el valor de la derecha a la caja de la izquierda.

```
nombre = "Tamagotchi-Bot"
bateria = 100

print(nombre)
```

Reglas para Nombrar Variables

No podemos ponerle cualquier nombre a nuestras cajas.

- **NO pueden tener espacios.** (Usa guion bajo _, esto se llama *snake_case*).
- **NO pueden empezar con números.** (`robot1` sí, `1robot` no).
- **Cuidado con las mayúsculas.** (`Bateria` y `bateria` son dos cajas distintas).

Ejemplos

```
# Nombres correctos:
vida_maxima = 100
sensor_frontal = "Activo"
```

Tipos de Datos Elementales

No es lo mismo la palabra ”10”que el número 10.

- **Strings (`str`):** Textos y letras. Siempre entre comillas .
- **Enteros (`int`):** Números puros. Sin comillas.
- **Booleanos (`bool`):** Verdadero (`True`) o Falso (`False`). Son la base de la lógica. Muy útiles para ”switches”.

Conversión Crítica con `input()`

Todo lo que entra por `input()` es texto. ¡Para matemáticas, usa `int()`!

```
# Si el usuario escribe 10, guarda un número 10 real:
edad = int(input("Ingresa la edad del robot: "))
```

F-Strings (Formateo Elegante)

Para inyectar el valor de las variables directamente dentro de un texto, usamos la letra `f` antes de las comillas y llaves $\{\}$ para las variables.

```python
nombre = "Wall-E"
bateria = 85

print(f"Hola {nombre}. Tu nivel de bateria es {bateria}%.")
```

Ejemplos Prácticos: Fundamentos

Entradas y Salidas Compuestas

```python
# Imprimir multiples palabras separadas por comas automaticamente:
print("Estado del robot:", "Optimo", "-", "Sensores online")

# Leer y convertir al mismo tiempo
energia = int(input("¿Cuanta energia quieres inyectar? "))
print(f"Recibidas unidades de energia: {energia}")
```

Matemáticas dentro del Print

Las f-strings operan la matemática antes de imprimirla:

```python
vida_base = 100
print(f"La vida con bonificacion es {vida_base + 50}")
```

Hoja de Ruta del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

Operadores Matemáticos y Relacionales

- **Suma (+) y Resta (-):** Modifican valores (ej. `bateria = bateria - 20`).
- **Comparadores:** Resultan en un Booleano (`True/False`). Menor/Mayor <, >.
- **Igualdad:** Un solo igual = asigna o guarda cosas. ¡Dos iguales == comparan!

```
bateria = 10
print(bateria == 0) # Imprimira False
```

Operadores Lógicos (`and`, `or`, `not`)

Para tomar decisiones más complejas, unimos preguntas:

- `and` (Y): **TODO** debe cumplirse para ser verdadero.
- `or` (O): **AL MENOS UNO** debe cumplirse para ser verdadero.

```python
peligro = True
escudos = 0

# Si no quedan escudos y ademas hay peligro...
print(escudos == 0 and peligro == True)
```

Estructuras de Control Condicional: if, elif, else

Evaluamos booleanos para ejecutar bloques. Obligatorio el uso de indentación (espacios).

```
comando = input("[1] Jugar [2] Dormir > ")

if comando == "1":
    print("¡A jugar!")
elif comando == "2":  # Mas opciones exclusivas
    print("Apagando sensores oculares...")
else:                 # Atrapa cualquier otro caso (errores)
    print("Comando invalido.")
```

Ejemplos Prácticos: Lógica de Batalla

Lógica Compleja y Anidada en Batallas

```python
ataque_enemigo = 150
modo_defensa = True
escudos_restantes = 100

if ataque_enemigo > 100 and modo_defensa == False:
    print("¡Impacto critico! Danos severos detectados.")
elif ataque_enemigo > 100 and modo_defensa == True:
    print("Bloqueo exitoso. Pierdes todos los escudos.")
    escudos_restantes = 0
else:
    print("Ataque repelido sin problemas.")
```

Hoja de Ruta del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

Listas (`list`): La mochila del robot

Para organizar información usamos corchetes `[]`. El conteo de las posiciones en programación **siempre parte desde 0**.

```python
mochila = ["Tuerca", "Bateria", "Engranaje"]

print(f"Primer objeto: {mochila[0]}") # Imprime "Tuerca"
print(f"Tercer objeto: {mochila[2]}") # Imprime "Engranaje"
```

Propiedades de Listas: ¿De qué nivel es mi mochila?

Podemos saber automáticamente cuántos objetos hay en nuestra lista utilizando la función *length* o `len()`:

```
mochila = ["Tuerca", "Bateria", "Engranaje"]

cantidad = len(mochila)
print(f"Tengo {cantidad} objetos guardados.") # Imprime 3
```

Modificando Elementos Específicos

Si sabemos la posición exacta, podemos sobreescribir ese dato directamente y reemplazarlo por algo nuevo.

```python
mochila = ["Tuerca", "Bateria", "Engranaje"]

# Encontramos un Cable y botamos la vieja Bateria
mochila[1] = "Cable Brillante"

print(mochila)
# > ['Tuerca', 'Cable Brillante', 'Engranaje']
```

Ejemplos Prácticos: Listas Diversas

En las listas no solo podemos guardar palabras; soportan cualquier tipo de dato elemental que necesites recolectar.

Diferentes Colecciones

```
# Coleccion de numeros (e.g., lecturas de un sensor de distancia)
lecturas_cm = [15, 12, 10, 8, 2]

# Coleccion de booleanos (e.g., el estado de componentes [Motor,
    Brazo, Ojo])
estado_hardware = [True, False, True]

# Listas vacias (Añades elementos despues en el programa)
inventario_explorador = []
```

Hoja de Ruta del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

Ciclo `for`: Iteración definida

Nos ahorra líneas infinitas de desarrollo al recorrer todos los elementos de una colección (como nuestra mochila).

```python
mochila = ["Tuerca", "Bateria Vieja", "Engranaje"]

# Para cada objeto que exista en la mochila... haz lo siguiente:
for objeto in mochila:
    print(f"Analizando con scanner: {objeto}")
```

Ciclo `while`: Iteración indefinida

Funciona como el ”Motor”de un juego. Mantiene repitiendo el bloque de código **mientras** su test lógico entregue `True`.

```
bateria = 3

while bateria > 0:
    print(f"Motores encendidos. Nivel: {bateria}")
    bateria = bateria - 1    # Importante para no
        estallar el PC

print("Bateria agotada, motor detenido.")
```

Freno de Mano: `break` y `continue`

Dentro de los bucles, tenemos ”Botones de pánico.extra.

- `break`: Destruye el bucle y te saca de él instantáneamente de manera forzada.
- `continue`: Omite el resto del código actual y salta directo a la siguiente vuelta del bucle.

```
while True: # Un bucle que giraria infinitamente en teoria...
    comando = input("> ")
    if comando == "SALIR":
        print("Abortando mision...")
        break # Esto detiene el while infinito
```

Ejemplos Prácticos: Dinámicas con Ciclos

For: Cuenta Regresiva Espacial

```
conteo = [3, 2, 1]
for numero in conteo:
    print(f"Lanzamiento en... {numero}!")
print("¡DESPEGUE!")
```

While: Verificación de Contraseña

```
acceso_concedido = False
while acceso_concedido == False:
    clave = input("Ingresa la clave maestra del robot: ")
    if clave == "1234":
        print("Clave Correcta.")
        acceso_concedido = True
```

Hoja de Ruta del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

Ejemplos Prácticos: Vaciando Listas de a poco

Una técnica recurrente es usar ‘.remove()‘ repetidas veces en un juego o en robótica para purgar eventos.

Consumiendo recursos iterativamente

```python
# Si recibimos un cargamento de suministros identicos
inventario = ["Agua", "Comida", "Agua", "Comida"]

if "Agua" in inventario:
    print("Bebiendo la primera racion de agua...")
    inventario.remove("Agua")

# El inventario ahora es: ["Comida", "Agua", "Comida"]
# Solo se borra UNA instancia, la primera que encuentra.
```

Hoja de Ruta del Taller

1. Introducción a Python: Fundamentos y Sintaxis Básica
2. Lógica y Toma de Decisiones
3. Estructuras de Datos y Colecciones
4. Control de Flujo: Ciclos y Repetición
5. Conceptos Intermedios (Bases para el Mega Desafío)
6. Mega Desafío Final

El Mega Desafío Final: Crea tu Tamagotchi

Objetivo: Construir desde **CERO** tu propio Tamagotchi-Bot interactivo utilizando combinadamente todas las herramientas que aprendimos hoy.

Construiremos el proyecto iterativamente en 3 misiones de dificultad ascendente. ¡A programar!

Misión Bronce: El Despertar del Robot

Objetivo Bronce: Código Base

Construye la estructura neuronal básica de tu robot:

- Declara un `nombre` con `input()` y una `energia`.
- Crea un ciclo `while True` infinito que funcione como el motor del juego, imprimiendo su energía usando f-strings.
- Crea un menú interactivo con `input()` que ofrezca: [1] Jugar, [2] Dormir, [3] Salir.
- Usa la estructura `if / elif` para que ”Jugarreste energía, ”Dormir”sume, y ”Salirçorte el programa con un `break`.

Misión Plata: Física y Almacenamiento

Objetivo Plata: Límites e Inventario

El robot ya vive, pero se rompe si su energía baja de 0.

- **Límites Lógicos:** Usa **Lógica Anidada** (If dentro de bloque If) para forzar que, si la energía supera el 100 %, baje de golpe a 100 de nuevo.
- **Mochila de Exploración:** Declara una lista vacía `mochila = []` antes del while. Añade la opción `4. Explorar`. Pídele al usuario que escriba un objeto y guárdalo usando `.append()`.

Misión Oro: Código Secreto

Objetivo Oro: Easter Egg IEEE

Nadie sabe que los ingenieros de IEEE RAS dejaron una puerta trasera en el código fuente del robot.

Tu misión:

- Añade un `elif` secreto a tu menú principal que escuche la palabra exacta `ÏEEE"`.
- Si el usuario escribe `ÏEEE"`, imprime un mensaje de ”Modo Desarrollador”, sube la energía a 9999 de forma absoluta y agrega un objeto legendario a la mochila.

Sé parte de RAS

¡Escanea el código y únete a IEEE RAS!